NCAR
OPERATED BY UCAR

GDEX
Geoscience Data Exchange



UPDATED AUGUST 2026

dsquasar: Backing Up the GDEX Archive onto Quasar

Gather → List → Tar → Transfer, tracked end-to-end in GDEXDB

Zaihua Ji

Software Engineer • zji@ucar.edu

BACKUP & DISASTER RECOVERY • BATCH OPERATIONS • COMPANION UTILITY TO DSARCH

Tars are built through dsarch AQ, uploaded through dsglobus, and scheduled by dscheck — every step tracked in GDEXDB.

OVERVIEW

What is dsquasar?

dsquasar keeps a **tape copy of the whole GDEX archive**. It finds files never backed up, bundles them into large tar files with `dsarch AQ`, then uploads many tars at once to the Quasar end points with `dsglobus`.

Driven entirely by GDEXDB flags

- Nothing is configured per run: a dataset joins the backup when `dataset.backflag` is set; `dsgroup.backflag` refines it per group.
- Any file with `bid = 0` is not backed up yet, and is picked up automatically.
- Every tar gets a `bfile` record whose status tracks it from list to tape.
- Work is resumable: a run that stops leaves its pending records behind.

Gather

find Web/Saved files still to back up

List

write dsarch input files, add bfile records

Tar

bundle into >=5 GB tar files via dsarch

Transfer

dsglobus batches to gdex-quasar (+drdata)

Verify

compare sizes on tape, reset if wrong

Report

statistics & email for specialists

GET IT & LEARN IT

Install & Resources

GitHub Repository

Source code, issues, and contributions.

```
git clone https://github.com/
  NCAR/rda-python-dsquasar.git
```

→ github.com/NCAR/rda-python-dsquasar

PyPI Package

Install the released package with pip; pulls in rda_python_common and rda_python_dsarch.

```
pip install rda_python_dsquasar
```

→ pypi.org/project/rda-python-dsquasar

Built-in Help

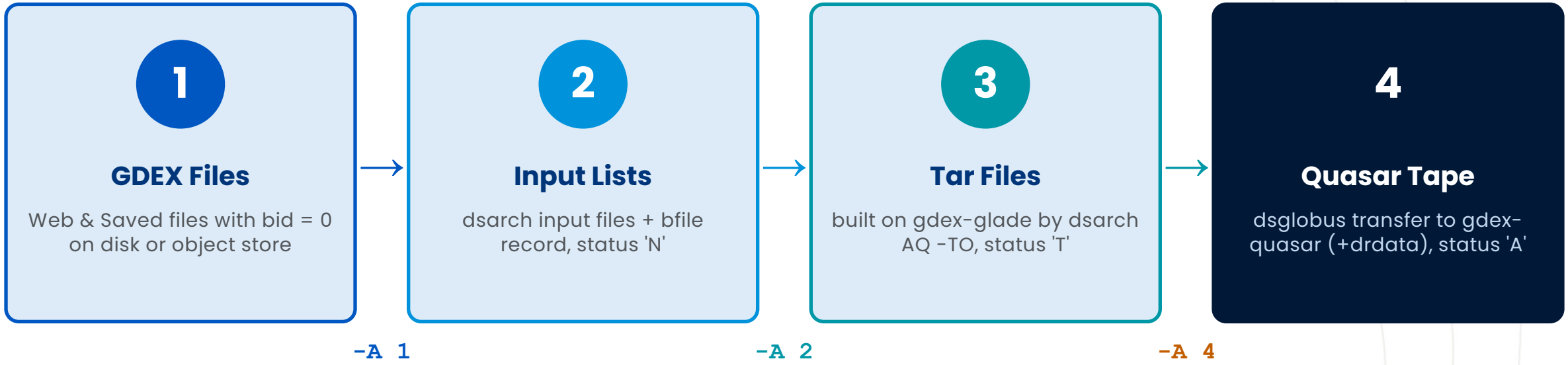
The full usage document ships with the package and is shown by -h.

```
dsquasar -h
# renders dsquasar.usg
```

→ src/rda_python_dsquasar/dsquasar.usg

one console command: `dsquasar` backs up the whole GDEX archive onto the Quasar end points.

The Four-Stage Backup Pipeline



-A 3 is the workhorse. It runs stages 1 and 2 together: files listed this run are tarred immediately, and existing `'N'` records left by an earlier run are picked up too. `-A 4` then moves the tars to tape. Use `-A 7` for the full chain in one run.

What dsquasar Reads and Writes in GDEXDB

Backup Flags

`dataset / dsgroup` **read**

- backflag 'B' — Backup copy only
- backflag 'D' — Backup + Drdata copy
- backflag 'N' — do not back up
- group flag 'P' inherits the dataset flag
- dataset.pid / lockhost — the run lock

Data Files

`wfile / sfile` **read + update**

- bid = 0 ⇒ not backed up yet
- bid = <bfile> ⇒ inside that tar
- data_size / checksum / timestamps re-verified on disk before tarring
- locflag 'O' files are pulled from object storage first

Backup File Record

`bfile` **write**

- one record per tar file
- status 'N' → 'T' → 'A'
- type 'B' or 'D'; dsids lists extra datasets
- note holds the whole input-file listing — the manifest of what is inside

The bfile note is the recovery key: it records every member file, its size and checksum, so a tar can be re-listed, re-tarred, or restored without re-reading the archive.

Anatomy of a Command

```
dsquasar (-a | -t dNNNNNN…) [-A Bits] [-B|-D] [Run…] [-e|-E]
```

Scope

`required`

Which datasets. -a walks every dataset with a Backup flag; -t takes IDs and accepts the SQL wildcard %.

Action

`-A Bits`

What to do, as a bitmask: 1 list, 2 tar, 4 transfer, 8 check, 16 stats. Defaults to 3.

Copies

`-B | -D`

A filter, not an override. -B keeps single copy datasets, -D keeps two copy ones. Default: each dataset follows its own flag.

Run control

`-b -d -m -l -n -W -w -u`

How it runs: background, PBS delay, processes, locking, dry run, worker slots.

Notes either `-a` or `-t` must be given, otherwise the usage is shown • `-a` wins over `-t` with a warning • the run always chdir's to the shared Quasar work directory • `-h` prints the full usage document.

QUICK START

Six Commands That Cover Most Work

1 See what is waiting

Counts only — gathers and reports, performs no backup.

```
dsquasar -t d123456 -n
```

2 Back up one dataset

Create the input lists and build the tar files.

```
dsquasar -t d123456 -A 3
```

3 Send the tars to tape

Batches built tars into >=90 GB dsglobus transfers.

```
dsquasar -t d123456 -A 4
```

4 The nightly production run

All flagged datasets, as a PBS batch job with email.

```
dsquasar -a -A 3 -e -b -d PBS
```

5 Report on everything

Backup statistics for all datasets, mailed to the specialist.

```
dsquasar -a -A 16 -e
```

6 Clear a stale lock

Unlock datasets left locked by a crashed run.

```
dsquasar -t d123456 -u
```

Everything is incremental. A file is a candidate only while `bid = 0`, so re-running never redoes finished work.

Backup Flags: What Gets Two Copies

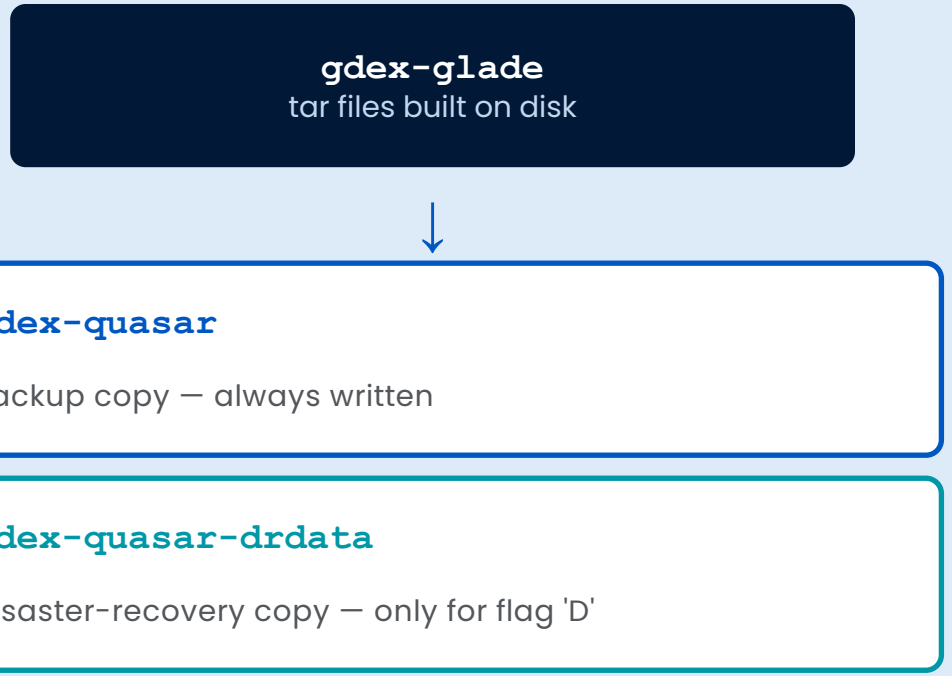
A dataset is put under Quasar backup by setting `dataset.backflag` — that one field is the switch, nothing else enrolls a dataset.

Flag	Value in dataset.backflag / dsgroup.backflag
`'B'`	Quasar Backup only — endpoint gdex-quasar
`'D'`	Backup + Drdata — also gdex-quasar-drdata
`'N'`	Not backed up at all
`'P'`	Group level only: inherit the dataset's flag

Group flags refine, they do not enroll. Once the dataset flag is set, a top-level `dsgroup` flag is applied group by group.

Set Backup Flag in the web config tool `gdex.ucar.edu/rda_pg_config/`
DSARCH → Dataset Information
sets `dataset.backflag` for the whole dataset
DSARCH → Group Information
sets `dsgroup.backflag` for one group
Per-run filter:
`-B` or `-D` limits a run to what is flagged that way; with neither, each dataset follows its own flag.

Two Globus end points, one source



A 'D' backup writes Drdata first, then Backup; both must succeed before the local tar is deleted.

The -A Action Bitmask

-A 1
Create Input

list Web/Saved files, add bfile records 'N'

-A 2
Tar Backup

build >=5 GB tars from 'N' lists → 'T'

-A 3
Create + Tar

the default, and the normal production run

-A 4
Transfer

dsglobus batches of 'T' tars to Quasar → 'A'

-A 6
Tar + Transfer

build then ship in one run

-A 7
Full chain

list, tar, and ship end to end

-A 8
Check

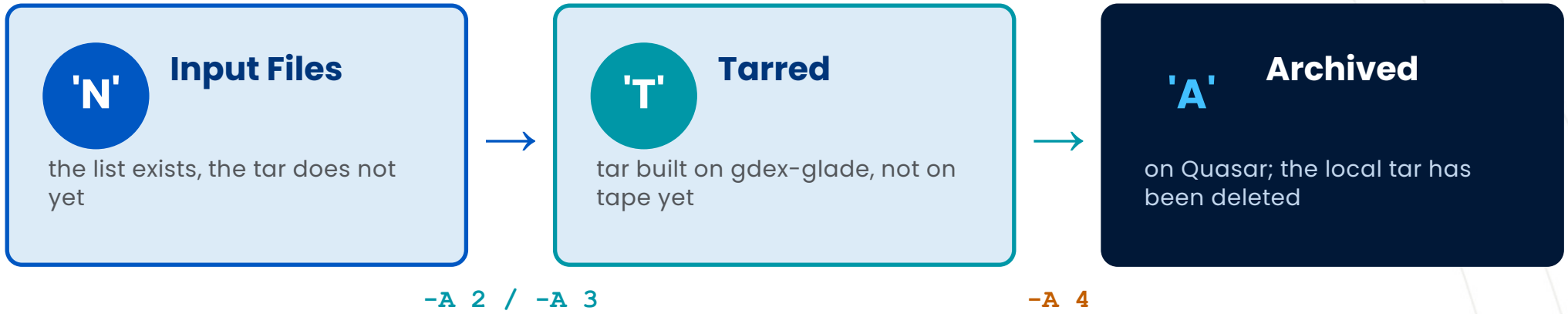
compare sizes on tape, reset bad ones to 'N'

-A 16
Statistics

dump counts for done and pending work

It really is a bitmask. 3 = 1+2, 6 = 2+4, 7 = 1+2+4 — only those combinations are accepted. **1**, **8** and **16** are single-process bookkeeping actions; **2**, **3**, **4**, **6**, **7** are the batch actions that get PBS cpus and multi-processing.

The bfile Status Lifecycle



Records also move backwards — that is the self-healing part

`'T' → 'N'`	the local tar file is missing when a transfer is prepared, so the tar is rebuilt from the stored input lists
`'A' → 'N'`	-A 8 finds the file absent or the wrong size on the end point, so the whole tar is redone
`record dropped`	dsarch reports every member file as already backed up elsewhere, so the duplicate placeholder is deleted

Because status drives everything, an interrupted run is never a problem: whatever is still `'N'` or `'T'` is simply picked up by the next run.

-A 1 Deep Dive: Building the Input Lists

For each flagged dataset

1. Lock the dataset, then re-gather its files under the lock
2. Skip files already listed in an open 'N' record
3. Re-check each file on disk or object storage; fix size, checksum and timestamp in GDEXDB
4. Append a line per file to a dsarch input file, per type
5. Close the group when big enough; add the bfile record

Input file layout

```
DS<=>d123456
WF<:>WT<:>SZ<:>MC<:>
file.nc<:>data<:>2147483648<:>3f1c…<:>
```

When is a tar big enough?

Threshold	Rule applied while accumulating files
20 GB	a single file alone is enough to be its own tar
5 GB	normal target size while under 100 files
2 GB	accepted minimum once 100 files have piled up
65 datasets	hard cap on datasets sharing one tar file

Small files are the reason. Tape wants few large files, so many tiny files are packed together — across datasets if need be — while a single very large file is never split.

Resulting tar file name

```
G001/d123456_sn40321_dn3_fn218.tar
```

subpath / dsid _ bid _ dataset count _ file count

ACTIONS

-A 2 Deep Dive: Building the Tar Files

```
dsarch d123456 AQ -QT D -QF G001/d123456_sn40321_dn3_fn218.tar -TO -OE -MD -IF d123456_W_40321.txt …
```

AQ	Archive Quasar — pack the listed files into one tar
-QT	copy type: 'B' Backup only, or 'D' Backup + Drdata
-QF	the tar file name recorded in the bfile record
-TO	Tar Only — build the tar, do not upload it
-OE	overwrite an existing Quasar file of that name
-MD	act on any specialist's dataset
-IF	the input files listing what goes into the tar

One child per tar file
With -m N, each tar file is handed to its own child process, so tarring scales widely across cpus.

Missing lists are rebuilt
The bfile note holds a copy of every input file, so a list deleted from the work directory is written back out before tarring.

Verified before it counts
After the children finish, the confirmed counts are re-read from GDEXDB, so the summary reports work that actually reached status 'T'.

Duplicates are dropped
If dsarch reports every member as already backed up under a different tar, the placeholder record and its lists are removed.

In an -A 3 run both phases tar. A list created this run is tarred right away, and any 'N' record left over from before is tarred too.

-A 4 Deep Dive: Transferring to Quasar

For each batch of tar files

1. Walk the 'T' records, accumulating tars until the batch hits 90 GB
2. Confirm each tar still exists on gdex-glade; if not, reset it to 'N'
3. Hand the whole list to dsglobus as one transfer — Drdata first, then Backup
4. Wait for both transfers to report finished
5. Delete the local tars and set every record to status 'A'

All or nothing. If either copy fails, no local tar is deleted and no record advances — the whole batch is retried by a later run.

Why batch to 90 GB?



18 tar files × 5 GB = one 90 GB dsglobus transfer

dsarch AQ can upload a tar as it builds it, but only one at a time. dsquasar calls dsglobus with the whole list, so a batch costs one request, not eighteen.

Source and destination paths

```
from   /data/gdex-quasar/d123456/<tar>       (Web)
       /decsdata/gdex-quasar/d123456/<tar>   (Saved)
to     /d123456/G001/<tar>
```

Single worker only. Transfers claim tars without dataset locks, so -A 4 never runs two workers.

-C CHANGEDAYS

Re-Backing Up Files That Changed

A file already on tape can be replaced on disk: a corrected version, a re-processed year. The tape copy is then stale. -c looks past the "never backed up" test and picks up files whose GDEXDB record changed recently.

```
dsquasar -t d123456 -A 1 -c 30
```

Only meaningful with -A 1 — the gather step is what decides which files land in the input lists. Must be greater than 0.

What changes

Selection	adds files with a recent change date, even if they already have a backup record
Tar name	gets a _changed suffix so the re-backup is obvious on tape
Old copy	stays on Quasar — nothing is deleted; the new tar is an additional version

The four-way gather

Each dataset is walked for up to four input lists, and -c widens each of them:

List	Contents
`Web / new`	Web files with no backup record
`Web / chg`	Web files changed within ChangeDays
`Saved / new`	Saved files with no backup record
`Saved / chg`	Saved files changed within ChangeDays

Pick ChangeDays to match your run cadence

A large window re-tars files that were already re-tarred on the previous run, burning tape for nothing. Roughly one gather interval plus a safety margin is the useful range.

Group flags still apply. A group with backflag 'N' is skipped even for changed files.

-A 8 Integrity Check • -A 16 Statistics

8 -A 8 — does tape match the database?

Walks every bfile record with status 'A' for the selected datasets, asks the Quasar end point for the real file size, and compares it against the size recorded in GDEXDB.

```
dsquasar -a -A 8 -e
```

`sizes agree`	record is left alone; the tape copy is trusted
`sizes differ`	status is reset 'A' → 'N' and the file is re-listed, re-tarred and re-sent on the next -A 3 / -A 4
`file missing`	same reset — the backup is simply redone from the archive

16 -A 16 — how much is on tape?

Reports, per dataset and as a grand total, what is already backed up and what is still waiting. No files are touched — it is a pure read of GDEXDB.

```
dsquasar -a -A 16 -E
```

Section	What it shows
`Backed up`	count and size already on Quasar
`Ready`	count and size still waiting
`Tar files`	bfile records by status: N / T / A
`Per dataset`	one line each, plus a summary row

Pair with -e or -E. Statistics are most useful mailed to the specialist.

-d PBS — Running as a Batch Job

Backing up the whole archive is far too big for a login session. -d turns the run into a delayed job: dsquasar writes a dscheck record and exits, and the dscheck daemon submits it to PBS when a slot is free.

```
dsquasar -a -A 3 -e -b -d PBS            # the usual cron line
```

1	**Cron fires**	dsquasar -a -A 3 -e -b -d PBS runs on the login host, in background mode.
2	**Size the job**	The pending tar count is turned into a cpu count, and PBS qoptions are written: walltime + select=1:ncpus=N:mem=Ngb.
3	**dscheck record**	A record is inserted keyed on command + specialist + argv + workdir. The argv is left unchanged, so the next identical cron line is recognised as a duplicate and blocks.
4	**Daemon submits**	dscheck picks the record up, submits to PBS, and dsquasar re-runs there — reading its process count back from the reserved ncpus.
5	**Retry on failure**	TRYLMTS['dsquasar'] = 3: a failed run is resubmitted up to three times before the record is reported as failed.

-d takes optional HostName and TryCount. -d PBS is the normal form; a bare -d uses the configured default batch host.

How Often It Runs in Production

Nobody starts a backup by hand. Four cron entries on cron.hpc.ucar.edu keep the archive current — each one an ssh into casper that leaves a dscheck record behind for PBS.

Cron time	dsquasar arguments	What the run does
`0 */4 * * *`	-a -A 3 -e -b -d PBS	Every 4 hours, six times a day — list the files still to back up and build the tars
`0 */4 * * *`	-a -A 4 -e -b -d PBS	Every 4 hours, six times a day — ship the finished tars to Quasar
`0 1 1 */3 *`	-a -c 1 -A 3 -l N -e -b -d PBS	1 AM on the 1st of every third month — re-back up files that changed
`0 3 1,15 * *`	-a -A 16 -e -b -d PBS	3 AM on the 1st and the 15th — dump the backup statistics

Why running that often is safe

Duplicates block

The cron line never changes, so dscheck recognises the argv and refuses a second submit while the first record is still pending.

Nothing is lost

A job killed at the walltime leaves its 'N' and 'T' records on file and its progress report in dscheck.einfo; the next run simply picks them up.

Build and ship overlap

-A 3 and -A 4 fire at the same minute. One tars what is ready while the other uploads what is already tarred, and the dataset lock keeps them off the same files.

Where the schedule lives. crontab/cron/zji_cron.cron in the rda-cron-controls repository; every line logs to dssdb/log/cronjob.log, and -e mails the report to the specialist.

SCHEDULING

Watching a Run with dscheck

-d hands the run to dscheck, so dscheck is also where it is watched — in the browser, or on the command line.

In the browser

gdex.ucar.edu/rda_pg_config/ → DSCHECK → Check Information, then search on Command = dsquasar.

```
# the same records from the command line
dscheck gc -cs -cm dsquasar
dscheck gc -cs -cm dsquasar -fn all
```

`-GC` lists the check records, `-CS` adds the live status line, `-CM` narrows the list to one command, and `-FN all` returns every field instead of the short set.

```
CheckIndex<:>ArgumentVector<:>Status<:>FileCount<:>DoneCount<:>
4105237<:>-a -A 3 -e -b -d PBS<:>Run 2H4M5S, 6% done PBS<5948540>/casper62<:>19195<:>1214<:>
4102650<:>-a -A 3 -e -b -d PBS -w 2<:>Run 23H37M58S, 2% done PBS<5928624>/casper64<:>54290<:>1193<:>
2 check records retrieved                                    # columns trimmed to fit
```

`Status`

How long the job has been running, how far along it is, and the PBS job id with the node it landed on. A failed check carries the error message here instead.

`FileCount / DoneCount`

The work in the record and the part of it finished, which is where the percentage comes from. Unchanged between two checks means the job is alive but not progressing.

`ArgumentVector`

The duplicate key. The second record differs only by -w 2, the extra worker slot, which is exactly why it gets a record of its own.

A long-running record is not by itself a problem. A first pass over a big archive legitimately runs for hours. The one to look at is the record whose DoneCount stops moving — it holds the duplicate key, so every later cron line blocks behind it.

-fn all — The Whole Check Record

The long form prints every column of the record. A few of them say more than the status line does.

```
CheckIndex<:>Command<:>ArgumentVector<:>Dataset<:>ActionName<:>Status<:>PBSQueue<:>ParentIndex<:>DownFlags<:>
FileCount<:>DoneCount<:>TryCount<:>MaxCount<:>DataSize<:>CheckDate<:>CheckTime<:>HostName<:>Specialist<:>
WorkDir<:>Modules<:>Environments<:>QSubOptions<:>ArgumenteXtra<:>ERrormessage<:>
```

Field	Value in the sample	What it tells you
ActionName	A3	Which -A action this record is running
TryCount / MaxCount	1 / 3	Submissions so far; a failed run is retried twice more
DataSize	227727328477	The bytes the run has to move
QSubOptions	-l walltime=24:00:00,select=1:ncpus=12:mem=12gb	What -d reserved: the walltime and the cpu count
ERrormessage	(empty)	Carries the failure text once a run gives up

ncpus is the auto-sizing, written down. ncpus=12 in QSubOptions is the process count dsquasar worked out from the pending tar count on the login host, stored in the record, and read back when the batch job starts. Reading it here is the simplest way to confirm what a run was actually given.

How Many CPUs Does the Job Ask For?

Tarring and uploading have very different shapes, so they are sized independently and the larger request wins — the phases run one after the other inside a single PBS allocation, sharing one ncpus.

Tarring (-A 2, -A 3)

One child per tar file — many small, cheap units.

```
MPTLIMIT = 200   Glade tar files per process
MPOLIMIT =  20   Boreas tar files per process
MPTMAX   =  12   process ceiling
procs = ceil(Glade/200 + Boreas/20)
```

A small divisor with a high cap: tar work spreads wide, Boreas tars ten times wider.

Uploading (-A 4)

One child per 90 GB batch — few large, network-bound units.

```
MPBLIMIT = 5000  tar files per process
MPBMAX   =    4  process ceiling
```

A large divisor with a low cap: more Globus streams do not move more bytes.

Why Boreas tars get more processes

Every member of a Boreas tar is staged down from the object store and checksummed before it is tarred, so it takes many times longer than a Glade tar. The backlog sat in one dataset, d694517 (~730 GB, ~146 tars): one process per 200 tars gave it 1, per 20 it gets 8.

-m sets the count by hand. With no -m, a delayed run sizes itself and reads the count back from the reserved PBS ncpus. An explicit -m always wins — including -m 1 to force a single process.

Making Sure the Report Outlives the Job

The queue gives the job 24 hours, and a full pass over the archive can want more. Cutting the work short to make time for the mail wastes a whole run, so instead dsquasar parks a progress report in its dscheck record an hour before the limit, and keeps working.



Ask PBS, not the clock	qstat gives back the walltime actually granted — the daemon can cap the 24 hours asked for to the queue maximum. The cutoff is always one hour short of it.
At the cutoff	A progress report is written straight into dscheck.einfo, naming the elapsed time and the tar files left to build and to transfer. The run does not stop.
If PBS kills the job	The daemon unlocks the dead record, finds einfo filled in, mails it and clears it. An overrun is reported instead of silent.
If the run finishes	The full report overwrites the cached one in einfo, so exactly one mail goes out — the complete one.

The guard only runs inside a real batch job under dscheck with -e; a foreground run mails its report directly.

WORKER SLOTS

-W / -w — When One Batch Job Hangs

The -A 3 gather runs from cron several times a day, so a crashed job restarts quickly — dead locks are taken over and dscheck retries. The gap was a job that is alive but too slow to finish: it holds the duplicate key and every later cron line simply blocks.

Stall test

A repeat submit inspects the running job. Either way counts as stalled:

```
runtime > WORKGRACE (1 hour) and
   done rate x walltime < MINWPROJ (150 %)
runtime > GATHGRACE (2 hours) and dcount = 0
```

A job that is not well on its way to finishing its recorded work this attempt is treated as stalled, and the next cron submit adds an extra worker alongside it — up to -W MaxWorkers, default 2.

```
dsquasar -a -A 3 -b -d PBS -W 2
```

Pinned to a single worker when locking is off. Extra workers rely on dataset locks to stay off each other's files, so -I N and the transfer actions (-A 4, 6, 7) always run one worker.

Slot numbering and direction

-w N is appended to the extra worker's own command line. Since argv is the only differing key field, that gives it a dscheck record of its own instead of colliding with slot 1.

Slot	Behaviour
`slot 1`	plain argv — repeat cron still blocks as a duplicate
`odd`	Boreas datasets first, then Glade; dsid ascending
`even`	Glade datasets first, then Boreas; dsid descending

A pair of workers starts on different storage and meets in the middle rather than fighting over the same dataset lock.

One huge dataset needs more processes. Locks are per dataset, so a second worker cannot help inside it; more tar processes (MPOLIMIT) do.

Keeping Two Runs Off the Same Files

Every mutual exclusion in dsquasar is a dataset lock — a pid and host stamped on the dataset record, claimed atomically and taken over automatically when the owning process is dead. There is no lock at the tar-file level.

Lock, then work

The dataset is locked before its files are gathered or its tars are built, and unlocked as soon as that dataset is finished.

Re-check under the lock

Work is listed before the lock is held, so the list can be hours old. Each candidate is re-read under the lock — still status 'N'? — before a tar is spent on it.

Multi-dataset tars

One tar may hold files from many datasets. All are locked before tarring and released once it is dispatched; if any is held elsewhere, the tar waits.

Tar names are safe

The predicted bid is only a guess. The insert uses AUTOID and the tar is renamed if the database returns a different id.

Operator controls

Option	Effect
`-l Y | -l N`	Lock datasets before backing them up. Y is the default; N skips locking entirely and forces a single worker.
`-u`	Clean up backup locks left behind on the named datasets. Dataset IDs are required — not valid with -a.

Locks left by a killed process are reclaimed automatically; -u is for the cases where you want them gone now.

-e / -E — Mail to the Specialist

A batch run leaves no one watching the screen, so dsquasar mails the outcome to the dataset specialist. Two levels of detail.

-e Summary

Totals only: how many files and bytes were backed up, and how much is still waiting.

-E Detailed

The same totals plus a per-dataset breakdown — which datasets moved, which are still behind.

Typical summary mail

```
Subject: dsquasar backup statistics

Backed up : 148,392 files   612.4 GB   in 128 tar files
Ready     :  22,105 files    91.7 GB
Tar files : N = 18   T = 6   A = 128

d123456   12,004 files   58.2 GB  backed up      2,110 files ready
d999001    1,338 files    9.9 GB  backed up          0 files ready
```

A run that overruns still mails. An hour before the PBS walltime the report so far is cached in dscheck.einfo, so even a job killed by the queue is reported rather than silent.

REFERENCE

Option Reference

Selection & scope	Meaning
`-a`	all datasets with a backup flag set
`-t ids`	named datasets; SQL % wildcard allowed
`-B`	only work flagged 'B' (Backup only)
`-D`	only work flagged 'D' (Backup + Drdata)
`-A bits`	actions: 1 2 3 4 6 7 8 16 (default 3)
`-c days`	with -A 1, re-back up recently changed files
`-e / -E`	mail summary / detailed statistics

Run control	Meaning
`-b`	background mode, no screen output
`-d [h] [n]`	delay mode: submit as a PBS batch job
`-m N`	concurrent child processes (default 1)
`-l Y | N`	lock datasets before backing up (default Y)
`-n`	show available file counts only, do nothing
`-W N`	max concurrent batch workers (default 2)
`-w N`	worker slot; set automatically, not by hand
`-u`	clean up backup locks on the named datasets
`-h`	display the help document

Either -a or -t is required — without one, dsquasar prints its usage and exits. -B and -D only narrow what a run processes; with neither, each dataset is backed up per its own backflag.

Six lines that cover most days

```
dsquasar -a -A 3 -e -b -d PBS     dsquasar -t d123456 -A 4
dsquasar -t d123456 -n            dsquasar -a -A 16 -e
dsquasar -t d123456 -A 1 -c 30    dsquasar -t d123456 -u
```

ENVIRONMENT

Where Things Live

dsquasar always runs from the Quasar backup work directory — it changes there itself, so it can be launched from anywhere.

```
$GDEXWORK/$COMMONUSER/quasar_backup
```

Work directory	Contents
`<dsid>_<t>_<bid>.txt`	dsarch input list, one per type per tar
`<dsid>_sn..._fn....tar`	the built tar, deleted after transfer
`dsquasar.log`	run log — first place to look
`dscheck record`	batch state, retries and progress counts

Globus & paths	Role
`gdex-glade`	source end point (archive on disk)
`gdex-quasar`	Backup copy on tape
`gdex-quasar-drdata`	disaster-recovery copy
`/data/...`	Web file staging path
`/decsdata/...`	Saved file staging path

Where the backup flags are set

`dsquasar` ships with rda_python_dsquasar. Backup flags are edited at `gdex.ucar.edu/rda_pg_config/` (DSARCH → Dataset Information).

TROUBLESHOOTING

When Something Looks Wrong

Nothing is gathered	The dataset or group backflag is not set. Check with `dsquasar -t <dsid> -n` first.
A repeat submit does nothing	The duplicate check working: an identical command is already queued or running.
Records stuck at 'N'	Tarring never reached them — the 23-hour guard, or the dataset was locked. Resolves next run.
Records stuck at 'T'	Built but not transferred. Run `-A 4` . If the local tar is gone the record resets to 'N'.
Sizes disagree with tape	Run `-A 8` — mismatched records reset to 'N' and are redone from the archive.
Locks left on a dataset	Dead-process locks are taken over automatically. To clear now: `dsquasar -t <dsid> -u`

Statuses only ever move forward on success. Every backwards step — T→N, A→N — is deliberate, and just means the work is queued to be done again.

RECOVERY

Getting Data Back Off Tape

Backing up is only half the point. Because every tar is recorded in GDEXDB with the datasets and files it contains, recovery is a lookup rather than a search of the tape.

Restore through dsarch

dsquasar writes the backup; dsarch reads it back. The -RQ action pulls a tar from Quasar and unpacks the requested files.

```
dsarch <dsid> -RQ ...
```

The bfile record supplies the tar name, the end point and the path, so the operator does not have to know where on tape the file sits.

Two independent copies

A dataset flagged D is written to both end points, Drdata first, so an interrupted run still leaves the DR copy complete.

End point	Copy
gdex-quasar	primary Backup copy
gdex-quasar-drdata	disaster-recovery copy

No separate recovery tool

Nothing extra to install: the same dsarch that stages files for the backup also reads them back. Point -RQ at a dataset and file, and it fetches the right tar from the right end point and unpacks only what you asked for.

Verify before you need it. A scheduled `-A 8` run is what turns “we have backups” into “we have backups that match the archive”.

RECOVERY

dsarch -RQ — How a File Comes Back

Name the files you want back and dsarch does the rest. The outline below is enough to run a restore; the dsarch guide covers the options in full.

```
dsarch <dsid> -RQ -WF <web files>   [-WT <types>]
dsarch <dsid> -RQ -SF <saved files>  -ST <types>     # or -QF <tar> for a whole tar
```

1	**Name the files**	-WF for web files, -SF plus -ST for saved files. -QF names a Quasar tar directly, and pulls it down without restoring anything.
2	**Find the tar**	Each wfile and sfile record carries the bid of the backup it went into, so one lookup gives the tar name, its end point and its path.
3	**Fetch it once**	The tar is transferred from gdex-quasar into the dataset work area. A tar already sitting there is reused, so several files out of one tar cost one transfer.
4	**Unpack what was asked**	The member list in the bfile note locates each file inside the tar — including any later rename — and only those members are extracted.
5	**Put it back**	Web files return to the dataset data tree or the object store, saved files to decsdata. Files still present on GDEX are left untouched.

Restores are safe to repeat. Anything already on GDEX is skipped, and each restored file is size checked against its GDEXDB record.

COLD STORAGE

Saved Files Now Go to GLADE Tape

Saved files under the decsdata area are no longer shipped to Quasar. They are handed to the GLADE HSM instead, which keeps them on the campaign file system until the batch processes migrate them onto tape.

Paused, not removed. Everything dsquasar does with Saved files still works — the `sfile` gathering, tarring and `-RQ` restore paths are untouched. Set the backup flags again and Saved files go back onto Quasar.

<decsdata>/<dsid>

The Saved files as dsarch stores them

→

cold_storage_<date>/<dsid>

decsdata_storage moves the dataset here

→

COLD_STORAGE/<dsid>

glade_hsm migrate -f relocates it once more

Run it

```
decsdata_storage -d d612000 d627000 -x
decsdata_storage -D 20250529 -w /gdex/decsdata -x
```

The HSM migrates every file of 100 MB or more onto tape, and everything under a `COLD_STORAGE/` path is marked immutable. No `bfile` records are written — cold storage is outside the dsquasar bookkeeping.

Options

Option	decsdata_storage — part of rda-python-miscs
`-D <date>`	Names cold_storage_<date>; defaults to today
`-w <dir>`	The decsdata directory; defaults to /gdex/decsdata
`-d | -l`	Dataset IDs on the command line, or a list file
`-x`	Mandatory — without it the help is shown

decsdata_restore — Three Steps Off Tape

A recall is only a request: the HSM reads the tape later, on its own schedule. Restoring is therefore always three commands. Name a dataset, or a sub-path of one, and without -D every cold storage directory is searched, the newest first.

```
decsdata_restore -d d612000/2020 -x      # 1. ask for it
decsdata_restore -d d612000/2020 -s      # 2. wait for it
decsdata_restore -d d612000/2020 -r      # 3. copy it back
```

-x **Request the recall**

Submits the glade_hsm recall request. First the restore size is added up from Table sfile in RDADB and compared with the free space gladequota reports — twice the size is required, since the recalled copy and the copy made by -r both live under the decsdata quota.

-s **Watch the status**

Reports how many files are still on tape, plus the log of any outstanding request. Repeat it until nothing is left offline.

-r **Copy it back**

Copies the recalled data back under <decsdata>/<dsid>, or under a target directory given after -r; a restored sub-path keeps its relative position. -f copies even while files are still on tape.

Seven-day window. Recalled files stay readable for 7 days and are then migrated back onto tape. -r leaves the cold copy in place, so taking a dataset out of the HSM for good means moving it out of the COLD_STORAGE path.

GDEX
Geoscience Data Exchange

NSF | NCAR
OPERATED BY UCAR

RECAP

Six Things Worth Remembering

1. **dataset.backflag enrolls**
   Setting that one field puts a dataset under backup; dsgroup.backflag refines it. -t narrows a run, it does not override a flag.

2. **-A 3 is the working default**
   Create the input lists and build the tars in one pass. -A 4 ships them. -A 7 does both.

3. **Status is the whole story**
   N listed, T tarred, A archived. Backwards moves are deliberate and simply requeue the work.

4. **Tar size is the economics**
   5 GB target tars batched into 90 GB transfers is what makes tape and Globus efficient.

5. **The dataset lock is the only lock**
   Everything about concurrency — workers, re-checks, multi-dataset tars — is built on it.

6. **Nothing is lost by stopping**
   The 23-hour guard, the retry limit and the resumable statuses mean an interrupted run just continues later.

GDEX
Geoscience Data Exchange

NSF | NCAR
OPERATED BY UCAR

Any Questions?

Thank you — dsquasar keeps a tape copy of the whole GDEX archive, run after run.

Zaihua Ji • zji@ucar.edu • `dsquasar -h` • dsquasar.usg • github.com/NCAR/rda-python-dsquasar